

Division by Zero

A smart contract is vulnerable to division-by-zero if it performs an arithmetic operation (such as division or modulo) where the divisor is not guaranteed to be non-zero and can be influenced by external input, attacker-controlled state, or unpredictable environment conditions. By triggering a division-by-zero error, an attacker can force a contract to revert its logic at critical points. This can lead to severe Denial-of-Service (DoS) conditions, such as blocking withdrawals, halting token transfer flows, or preventing liquidation logic, auctions, and reward distributions.

Specification primitives Our defect specifications rely on standard Solidity datatypes— **Contract**, **Method**, **Address**, **Env(ironment)**, and **State**. A brief description of the necessary predicates and functions over these types is included in Table 1. The evaluation of these primitives is stratified: structural properties are derived statically via Slither, while behavioral properties are verified at runtime via Forge/Kontrol assertions within the LLM-generated test harness.

Table 1. Primitives for Division by Zero Specification

Symbol	Semantics
<i>Static Analysis (Slither)</i>	
$\text{methods}(c)$	The set of public/external methods defined in contract c .
$\text{Influence}(U, V)$	Holds if the value V is mathematically dependent on inputs provided by an external user U . It is used to identify “control gaps” in arithmetic operations.
$\text{Logic}(\phi)$	Represents the set of all semantic guards and validation logic implemented within the contract that are intended to enforce the truth of the predicate ϕ along a specific execution path.
<i>Runtime Verification (LLM-generated based on specific template)</i>	
$\text{IsPanic}(\sigma)$	Holds if a state transition resulted in a “Panic” revert (Solidity 0.8.x), which corresponds to internal logic errors (e.g., assert failure or array out-of-bounds) rather than user-side input validation (require).

Domain Conceptual Model. A smart contract c is vulnerable to division-by-zero if:

1. c performs an arithmetic operation x/y or $x \bmod y$; and
2. the divisor y is not guaranteed to be non-zero, and
3. the value of y may be controlled by external input, attacker-controlled state, or unpredictable environment conditions.

Formal Specification.

$$\begin{aligned} \forall m \in \text{methods}(c). \mathcal{D} : \text{Args} \times \text{State} &\rightarrow \mathbb{Z}. \\ \text{DivZero}(c) \equiv \exists \text{args}. \text{Influence}(U, \mathcal{D}(\text{args})) \wedge &(\mathcal{D}(\text{args}) = 0) \wedge \\ \neg \exists \mathcal{G} \in \text{Logic}(\mathcal{D} \neq 0) & \end{aligned}$$

where

$\mathcal{D} : \text{Args} \times \text{State} \rightarrow \mathbb{Z}$ defines the denominator function. This function is a mathematical abstraction used to model how the divisor in an arithmetic operation is computed within a specific smart contract method.

This identifies the control gap specifically when a denominator is computable from user input or fuzzed state and lacks a “require” check, leading to a potential DoS on critical arithmetic.

Specific test template design.

Testing Goal Identify if an external actor can influence a denominator to zero without passing through a semantic guard, such as a require statement.

Setup The LLM identifies arithmetic operations where the denominator is derived from user-supplied fuzzed parameters or mutable contract state variables.

Control Analysis The harness leverages symbolic execution via Kontrol to solve for the path constraint `DenominatorExpression == 0`, attempting to find a concrete value that triggers the fault.

Action If the symbolic solver identifies a satisfying input, the harness attempts to execute the target method m in Forge using those specific inputs to confirm reachability.

Assertion The test employs a `try/catch` block or `vm.expectRevert()` to intercept the specific Solidity Panic code 0x12 (division by zero). If the revert occurs at the division operation rather than a preceding guard, a Control Gap is formally flagged.

Template Derivation and LLM Instantiation The conceptual test design is materialized into a standardized Foundry test template. To bridge the gap between static analysis and generation, the template embeds explicit `[LLM_INSTRUCTION]` comments to guide the LLM through the instantiation phase. The process unfolds as follows:

Target Initialization (Setup): The LLM is instructed to import the target smart contract artifact and initialize the contract under test. It also imports `console2` to aid in debugging if a crash is found. The framework identifies arithmetic operations where the denominator is derived from fuzzed parameters or mutable state variables.

Input Parameterization (Fuzzing/Setup): To support fuzzing, the LLM analyzes the target method and adds necessary arguments to the test function signature. It uses `vm.assume` to mathematically constrain these inputs, and sets up any mutable contract state required to reach the vulnerable code using setters.

Control Analysis: The LLM is instructed to rely on the symbolic solver (Kontrol), which attempts to find specific inputs where the denominator expression evaluates to zero. If a satisfying assignment is found, the test proceeds.

Exploit Simulation (Action): The LLM synthesizes the action phase by executing the target method with the identified/fuzzed inputs to confirm the reachability of the division-by-zero fault. This is done by wrapping the call in a `try/catch` block.

Standardized Validation (Assertion): The test framework validates the defect by catching the specific Solidity Panic code `0x12` (Division or Modulo by Zero). If the execution succeeds without reverting, the test passes silently. However, if the `0x12` panic is caught, it prints debugging logs and explicitly forces a revert to stop the fuzzer and flag the confirmed Control Gap. Unrelated errors are caught and ignored to prevent false positives.

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity 0.8.29;
3
4  import {Test, stdError} from "../lib/forge-std/src/Test.sol";
5  import {console2} from "forge-std/console2.sol";
6  // [LLM_INSTRUCTION]: Import the artifact of the contract being tested. The
   solidity files are in "../src/". The name of the file is the same as the
   name of the contract.
7
8  // -----
9  // [Testing Goal] Identify if an external actor can influence a denominator
   to
10 // zero without passing through a semantic guard (e.g., a require statement).
11 // This identifies the Control Gap, when a denominator is computable from
12 // input or fuzzed state and lacks a "require" check, leading to potential
13 // DoS on critical arithmetic.
14 // -----
15
16 // [LLM_INSTRUCTION]: Name the contract 'TestDivisionByZero[ContractName]'
17 contract TestDivisionByZeroTemplate is Test {
18     // [LLM_INSTRUCTION]: Use StdStorage if needed for complex state setup:
19     // using stdStorage for StdStorage;
20
21     // -----
22     // [Setup] Declare the contract under test. Identify arithmetic
   operations
23     // where the denominator is derived from user-supplied fuzzed parameters
   or
24     // mutable contract state variables.
25     // -----
26
27     // [LLM_INSTRUCTION]: Declare the contract under test variable
28     // DivideByZeroMinimal public _contractUnderTest;
29
30     // ----- [Setup]
31

```

```

32     function setUp() public {
33         // -----
34         // [Setup] Initialize the contract under test
35         // -----
36
37         // [LLM_INSTRUCTION]: Initialize the contract under test.
38         // _contractUnderTest = new DivideByZeroMinimal();
39
40         // ----- [/Setup]
41     }
42
43     // [LLM_INSTRUCTION]: Analyze the method being tested.
44     // 1. If it accepts arguments, ADD them to this function signature to
45     //    enable Fuzzing.
46     // Example: function test_highlightThrowsDivisionByZeroException(uint256
47     //    amount) public {
48     function test_highlightThrowsDivisionByZeroException() public {
49         // -----
50         // [Setup] Constrain fuzz/symbolic inputs and configure pre-
51         //    conditions.
52         // -----
53
54         // [LLM_INSTRUCTION]: Use vm.assume to constrain fuzz parameters if
55         //    necessary.
56         // vm.assume(amount <= type(uint128).max);
57
58         // [LLM_INSTRUCTION]: Set up the state required to reach the
59         //    vulnerable code.
60         // _contractUnderTest.setVal(fuzzArg);
61
62         // ----- [/Setup]
63
64         // -----
65         // [Control Analysis] Leverage symbolic execution (Kontrol) to solve
66         //    for the path constraint DenominatorExpression == 0, attempting to
67         //    find a concrete value that triggers the fault.
68         // -----
69
70         // [LLM_INSTRUCTION]: The symbolic solver will attempt to find inputs
71         //    where the denominator evaluates to zero. If a satisfying
72         //    assignment
73         //    is found, the test proceeds to execute the target method with
74         //    those
75         //    specific inputs.
76
77         // ----- [/Control Analysis]
78
79         // -----
80         // [Action] Execute the target method with the identified inputs to
81         //    confirm reachability of the division-by-zero fault.
82         // -----
83
84         // [Assertion] Use a try/catch block to intercept the specific
85         //    Solidity
86         //    Panic code 0x12 (division by zero). If the revert occurs at the
87         //    division operation rather than a preceding guard, a Control Gap is
88         //    formally flagged.
89         // -----

```

```

83     try _contractUnderTest.methodName(/* [LLM_INSTRUCTION]: Replace
      methodName with the vulnerable method and insert its arguments
      here */) {
84         // [LLM_INSTRUCTION]: Case: Execution Succeeded.
85         // If the test MUST fail on division by zero, we do nothing here
          (pass).
86     } catch Panic(uint256 errorCode) {
87         // Panic Code 0x12 = Division or Modulo by Zero
88         if (errorCode == 0x12) {
89             console2.log("
              -----");
90             console2.log("[!] DIVISION BY ZERO FOUND (Control Gap)");
91             // [LLM_INSTRUCTION]: Log relevant variables to debug the
              crash
92             // console2.log("Input Amount:", amount);
93             console2.log("
              -----");
94
95             // [LLM_INSTRUCTION]: Force the test to fail.
96             // This stops the Fuzzer and displays the logs immediately.
97             revert("Division by Zero Detected - Control Gap flagged (Logs
              Printed)");
98         }
99     } catch {
100         // [LLM_INSTRUCTION]: Catch other unrelated errors (e.g.,
          standard reverts) and ignore them.
101     }
102
103     // ----- [/Action]
104     // ----- [/Assertion]
105 }
106 }

```

Listing 1.1. Division by Zero Multi-Shot test template